

计算机与信息工程学院实验报告

姓名	—	学号	
专业		年级	
课程	操作系统	主讲教师	刘扬
实验时间（年月日时）	2025 年 10 月 23 日 10:00-12:00		
实验地点	计算机大楼 313	辅导教师	刘扬
实验题目	进程通信		
实验目的	了解和熟悉 Linux 支持的消息通讯机制及信息量机制。		
实验环境（硬件和软件）	ubuntu+gcc		

一、OS 课程实验

实验内容：

1、消息的创建，发送和接收。

使用系统调用 `msgget()`, `msgsnd()`, `msgrev()`, 及 `msgctl()` 编制一长度为 1k 的消息的发送和接收程序。观察上面的程序，说明控制消息队列系统调用 `msgctl()` 在此起什么作用？

2、共享存储区的创建、附接和段接。

使用系统调用 `shmget()`, `shmat()`, `sgmdt()`, `shmctl()`, 编制一个与上述功能相同的程序。

3、比较上述 1、2 两种消息通信机制中数据传输的时间。

实验步骤：

1、使用消息队列的系统调用 `msgget()`, `msgsnd()`, `msgrev()`, 及 `msgctl()` 编写消息的发送和接收程序。

2、使用共享存储区的系统调用 `shmget()`, `shmat()`, `sgmdt()`, `shmctl()` 编写消息的发送和接收程序。

3、调试并运行两个代码，并比较两种方式的结果和性能。

实验数据记录：

1、消息队列

代码：

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/msg.h>
#include <sys/ipc.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#define MSGKEY 75

struct msgform
{
    long mtype;
    char mtext[100];
}msg;

int msgqid;

void client()
{
    int i;
    msgqid = msgget(MSGKEY, 0777|IPC_CREAT);
    for(i=10; i>=1; i--)
    {
        msg.mtype = i;
        printf("(client)sent\n%d\n", msg.mtype);
        msgsnd(msgqid, &msg, sizeof(msg.mtext), 0);
        sleep(1); // 等待1秒，让服务器有时间接收()
    }
    exit(0);
}

void server()
{
    msgqid = msgget(MSGKEY, 0777|IPC_CREAT);
    do
    {
        msgrcv(msgqid, &msg, sizeof(msg.mtext), 0, 0);
        printf("(server)receive\n%d\n", msg.mtype);
    } while (msg.mtype != 1);
    msgctl(msgqid, IPC_RMID, 0);
}

int main()
{
    if(fork())
    {
        server();
        wait(0);
    }
    else
    {
        client();
    }
    return 0;
}

```

问题 输出 调试控制台 终端

运行结果:

```
[ygkldnherr@Ubuntu] - (~/.Documents/OS/实验4)
> cd "/home/ygkldnherr/Documents/OS/实验4/"
(client)sent
10
(server)receive
10
(client)sent
9
(server)receive
9
(client)sent
8
(server)receive
8
(client)sent
7
(server)receive
7
(client)sent
6
(server)receive
6
(client)sent
5
(server)receive
5
(client)sent
4
(server)receive
4
(client)sent
3
(server)receive
3
(client)sent
2
(server)receive
2
(client)sent
1
(server)receive
1
```

2、共享存储区

代码:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/shm.h>
#include <sys/ipc.h>
#include <sys/wait.h>
#define SHMKEY 75
shmid=shmget(SHMKEY,1024,0777|IPC_CREAT);
int shmid, i; addr=shmat(shmid,0,0);
int *addr; *addr=-1;
void client() if(fork())
{
    int i; SERVER();
    shmid = shmget(SHMKEY, 1024, 0777|IPC_CREAT);
    addr = shmat(shmid, 0, 0);
    for(int i = 9; i >=0; i--){
        while(*addr != -1);
        printf("(client)sent\n");
        *addr = i;
    }
    exit(0);
}
结果)
运行的结果和预想的完全一样。但在运行
约 0.1 秒才有响应。同样，之后 client 又需要等
void server(){
    shmid = shmget(SHMKEY, 1024, 0777|IPC_CREAT);
    addr = shmat(shmid, 0, 0);
    do{
        while(*addr == -1);
        printf("(server)receive\n%d\n", *addr);
        if(*addr!=0){
            *addr--;
        }
    }while(*addr);
    shmctl(shmid, IPC_RMID, 0);
}
全面的分析。
消息队列的建立比共享区的设立消耗的资源少
内存的映像,当然控制起来比前者复杂.如果每
势。
当消息队列和共享区建立好后,共享区的数据传
主软件进行控制和实现,需要消耗一定的 CPU 资
*addr = 消息的传递,自身就带有同步的控制.当等到消息
if(fork()){不借助其他机制进行同步,接受数据的一方
server();
wait(0);
}else{
    client();
}
return 0;
}

```

运行结果：

```
[ygtkldnherr@Ubuntu] - (~/.Documents/OS/实验4)
> cd "/home/ygtkldnherr/Documents/OS/实验4/"
(client)sent
(server)receive
9
(client)sent
(server)receive
8
(client)sent
(server)receive
7
(client)sent
(server)receive
6
(client)sent
(server)receive
5
(client)sent
(server)receive
4
(client)sent
(server)receive
3
(client)sent
(server)receive
2
(client)sent
(server)receive
1
(client)sent
(server)receive
0
```

问题讨论：

1、控制消息队列系统调用 `msgctl()` 在此起什么作用？

`msgctl()` 在这里的作用是删除消息队列，释放系统资源。**System V** 消息队列是内核级对象，有以下特点：持久性：即使创建它的进程退出，消息队列仍然存在，需要手动清理：不会自动删除，会一直占用系统资源，有数量限制：系统中消息队列数量有上限。如果不调用 `msgctl`，程序结束时不会删除消息队列，会一直占用系统资源，另外下次再运行程序时可能会产生错误。

二、OS 源码阅读

1. 源代码文件名称

`msg.c;sem.c;shm.c`

2. 源代码功能描述

msg.c

整体功能：实现了 **System V** 消息队列机制，包括消息队列的创建、发送、接收、控制和删除等功能。使用静态数组管理消息队列，支持进程间通过消息进行通信。

函数 `msg_init` 输入：无

函数 `msg_init` 输出：无

函数 `msg_init` 功能：初始化消息队列系统，清空所有消息队列数组，重置全局统计变量和序列号。

函数 `sys_msgsnd` 输入：

`msqid`：消息队列标识符

`msgp`：指向消息缓冲区的指针

`msgsz`：消息大小

`msgflg`：发送标志

函数 `sys_msgsnd` 输出：成功返回发送的字节数，失败返回错误码

函数 `sys_msgsnd` 功能：向指定消息队列发送消息。验证参数合法性，检查权限，分配消息内存，将消息加入队列尾部，并唤醒等待读取的进程。

函数 `sys_msgrcv` 输入：

`msqid`：消息队列标识符

`msgp`：指向接收缓冲区的指针

`msgsz`：接收缓冲区大小

`msgtyp`：期望的消息类型

`msgflg`：接收标志

函数 `sys_msgrcv` 输出：成功返回实际接收的字节数，失败返回错误码

函数 `sys_msgrcv` 功能：从消息队列接收消息。支持按类型过滤消息（精确匹配、排除匹配、小于等于匹配），处理消息截断，唤醒等待写入的进程。

函数 `findkey` 输入：`key`：消息队列键值

函数 `findkey` 输出：找到返回队列索引，未找到返回-1

函数 `findkey` 功能：根据键值在现有消息队列中查找对应的队列索引。

函数 `newque` 输入：

`key`：消息队列键值

`msgflg`：创建标志

函数 `newque` 输出：成功返回消息队列标识符，失败返回错误码

函数 `newque` 功能：创建新的消息队列。分配队列结构体，初始化权限信息和队列状态，生成唯一的序列号。

函数 `sys_msgget` 输入：

`key`：消息队列键值

`msgflg`：获取标志

函数 `sys_msgget` 输出：成功返回消息队列标识符，失败返回错误码

函数 `sys_msgget` 功能：获取或创建消息队列。处理 `IPC_PRIVATE` 特殊情况，检查 `IPC_CREAT` 和 `IPC_EXCL` 标志，验证用户权限。

函数 `freeque` 输入：`id`：消息队列索引

函数 `freeque` 输出：无

函数 **freeque** 功能：释放指定消息队列的所有资源。递增序列号避免标识符重用，释放所有排队消息，唤醒等待的读写进程。

函数 **sys_msgctl** 输入：

msqid：消息队列标识符

cmd：控制命令

buf：用户空间缓冲区指针

函数 **sys_msgctl** 输出：成功返回 0 或相关信息，失败返回错误码

函数 **sys_msgctl** 功能：消息队列控制函数。支持多种命令：

- **IPC_STAT**：获取队列状态信息
- **IPC_SET**：设置队列参数
- **IPC_RMID**：删除消息队列
- **IPC_INFO/MSG_INFO**：获取系统消息队列信息
- **MSG_STAT**：获取队列统计信息

sem.c

整体功能：实现了 **System V** 信号量机制，用于进程间的同步和互斥。支持信号量集的创建、操作、控制和撤销操作，实现了 **FIFO** 等待队列以避免进程饥饿。

函数 **sem_init** 输入：无

函数 **sem_init** 输出：无

函数 **sem_init** 功能：初始化信号量系统，清空信号量数组，重置全局变量和统计信息。

函数 **findkey** 输入：key：信号量集键值

函数 **findkey** 输出：找到返回信号量集索引，未找到返回-1

函数 **findkey** 功能：根据键值在现有信号量集中查找对应的索引。

函数 **newary** 输入：

key：信号量集键值

nsems：信号量数量

semflg：创建标志

函数 **newary** 输出：成功返回信号量集标识符，失败返回错误码

函数 **newary** 功能：创建新的信号量集。检查资源限制，分配内存，初始化权限信息和信号量状态。

函数 **sys_semget** 输入：

key：信号量集键值

nsems：信号量数量

semflg：获取标志

函数 **sys_semget** 输出：成功返回信号量集标识符，失败返回错误码

函数 **sys_semget** 功能：获取或创建信号量集。处理 **IPC_PRIVATE** 特殊情况，检查权限和信号量数量限制。

函数 **insert_into_queue / remove_from_queue** 输入：

sma：信号量集结构体指针

q: 队列元素指针

函数 insert_into_queue / remove_from_queue 输出: 无

函数 insert_into_queue / remove_from_queue 功能: 管理信号量操作的 FIFO 等待队列的插入和删除操作。

函数 try_semop 输入:

sma: 信号量集结构体指针

sops: 信号量操作数组

nsops: 操作数量

函数 try_semop 输出:

0: 操作可以立即执行

1: 需要等待

负数: 错误码

函数 try_semop 功能: 检查信号量操作序列是否可以原子性地执行, 使用回滚机制保证检查的原子性。

函数 do_semop 输入:

sma: 信号量集结构体指针

sops: 信号量操作数组

nsops: 操作数量

un: undo 结构指针

pid: 进程 ID

函数 do_semop 输出: 成功返回 0

函数 do_semop 功能: 实际执行信号量操作序列, 更新信号量值和 undo 信息。

函数 update_queue 输入: sma: 信号量集结构体指针

函数 update_queue 输出: 无

函数 update_queue 功能: 遍历等待队列, 检查哪些等待的操作现在可以执行, 并唤醒相应的进程

函数 count_semnrcnt / count_semzcnt 输入:

sma: 信号量集结构体指针

semnum: 信号量编号

函数 count_semnrcnt / count_semzcnt 输出: 返回等待进程计数

函数 count_semnrcnt / count_semzcnt 功能: 统计等待信号量值非零/为零的进程数量。

函数 freeary 输入: id: 信号量集索引

函数 freeary 输出: 无

函数 freeary 功能: 释放信号量集资源。使标识符无效, 唤醒所有等待进程并让它们失败, 释放相关内存。

函数 sys_semctl 输入:

semid: 信号量集标识符

semnum: 信号量编号

cmd: 控制命令

arg: 参数联合体

函数 sys_semctl 输出: 成功返回 0 或相关信息, 失败返回错误码

函数 sys_semctl 功能: 信号量控制函数。支持多种命令:

GETVAL/SETVAL: 获取/设置单个信号量值

GETPID: 获取最后操作进程 ID

GETNCNT/GETZCNT: 获取等待进程计数

GETALL/SETALL: 获取/设置所有信号量值

IPC_STAT/IPC_SET: 获取/设置信号量集状态

IPC_RMID: 删除信号量集

IPC_INFO/SEM_INFO: 获取系统信号量信息

函数 sys_semop 输入:

semid: 信号量集标识符

tsops: 信号量操作数组指针

nsops: 操作数量

函数 sys_semop 输出: 成功返回 0, 失败返回错误码

函数 sys_semop 功能: 执行信号量操作。验证参数和权限, 尝试执行操作, 如果无法立即执行则加入等待队列。支持 SEM_UNDO 机制防止进程异常终止时资源泄漏。

函数 sem_exit 输入: 无

函数 sem_exit 输出: 无

函数 sem_exit 功能: 进程退出时清理信号量相关资源。移除进程的等待队列项, 执行所有注册的 undo 操作, 释放 undo 结构。

shm.c

整体功能: 实现了 System V 共享内存机制, 允许不相关的进程通过共享内存区域进行高效通信。支持共享内存的创建、附加、分离、控制和交换。

函数 shm_init 输入: 无

函数 shm_init 输出: 无

函数 shm_init 功能: 初始化共享内存系统, 清空共享内存段数组, 重置全局统计变量。

函数 findkey 输入: key: 共享内存键值

函数 findkey 输出: 找到返回共享内存索引, 未找到返回-1

函数 findkey 功能: 根据键值在现有共享内存段中查找对应的索引。

函数 newseg 输入:

key: 共享内存键值

shmflg: 创建标志

size: 共享内存大小

函数 newseg 输出: 成功返回共享内存标识符, 失败返回错误码

函数 newseg 功能: 创建新的共享内存段。检查大小限制和系统资源, 分配段描述符和页表数组, 初始化权限信息。

函数 `sys_shmget` 输入:

key: 共享内存键值

size: 共享内存大小

shmflg: 获取标志

函数 `sys_shmget` 输出: 成功返回共享内存标识符, 失败返回错误码

函数 `sys_shmget` 功能: 获取或创建共享内存段。处理 `IPC_PRIVATE` 特殊情况, 检查权限和大小限制。

函数 `killseg` 输入: id: 共享内存索引

函数 `killseg` 输出: 无

函数 `killseg` 功能: 销毁共享内存段。释放所有物理页和交换空间, 更新系统统计, 释放段描述符内存。

函数 `sys_shmctl` 输入:

shmid: 共享内存标识符

cmd: 控制命令

buf: 用户空间缓冲区指针

函数 `sys_shmctl` 输出: 成功返回 0 或相关信息, 失败返回错误码

函数 `sys_shmctl` 功能: 共享内存控制函数。支持多种命令:

IPC_STAT: 获取段状态信息

IPC_SET: 设置段参数

IPC_RMID: 标记删除共享内存段

SHM_LOCK/SHM_UNLOCK: 锁定/解锁段在内存中 (仅超级用户)

IPC_INFO: 获取系统共享内存信息

SHM_INFO: 获取详细统计信息

SHM_STAT: 获取段统计信息

函数 `insert_attach / remove_attach` 输入:

shp: 共享内存段结构体指针

shmd: 虚拟内存区域结构体指针

函数 `insert_attach / remove_attach` 输出: 无

函数 `insert_attach / remove_attach` 功能: 管理共享内存附加描述符的双向循环链表。

函数 `shm_map` 输入: shmd: 虚拟内存区域结构体指针

函数 `shm_map` 输出: 成功返回 0, 失败返回错误码

函数 `shm_map` 功能: 建立共享内存的页表映射。清除旧映射, 创建新的页表项, 设置共享内存签名。

函数 `sys_shmat` 输入:

shmid: 共享内存标识符

shmaddr: 请求的附加地址

shmflg: 附加标志

raddr: 返回的实际附加地址

函数 `sys_shmat` 输出：成功返回 0，失败返回错误码

函数 `sys_shmat` 功能：将共享内存段附加到进程地址空间。处理地址对齐和映射，创建 `vm_area_struct`，建立页表映射。

函数 `shm_open` 输入：`shmd`：虚拟内存区域结构体指针

函数 `shm_open` 输出：无

函数 `shm_open` 功能：在 `fork` 时处理共享内存段的继承。将新的附加描述符插入段链表，更新附加计数。

函数 `shm_close` 输入：`shmd`：虚拟内存区域结构体指针

函数 `shm_close` 输出：无

函数 `shm_close` 功能：分离共享内存段。取消页表映射，从段链表中移除，如果段被标记删除且无附加则销毁。

函数 `sys_shmdt` 输入：`shmaddr`：共享内存附加地址

函数 `sys_shmdt` 输出：成功返回 0

函数 `sys_shmdt` 功能：从进程地址空间分离共享内存段。查找对应的 `vm_area_struct` 并取消映射。

函数 `shm_swap_in` 输入：

`shmd`：虚拟内存区域结构体指针

`offset`：页内偏移

`code`：交换代码

函数 `shm_swap_in` 输出：成功返回页表项，失败返回错误页

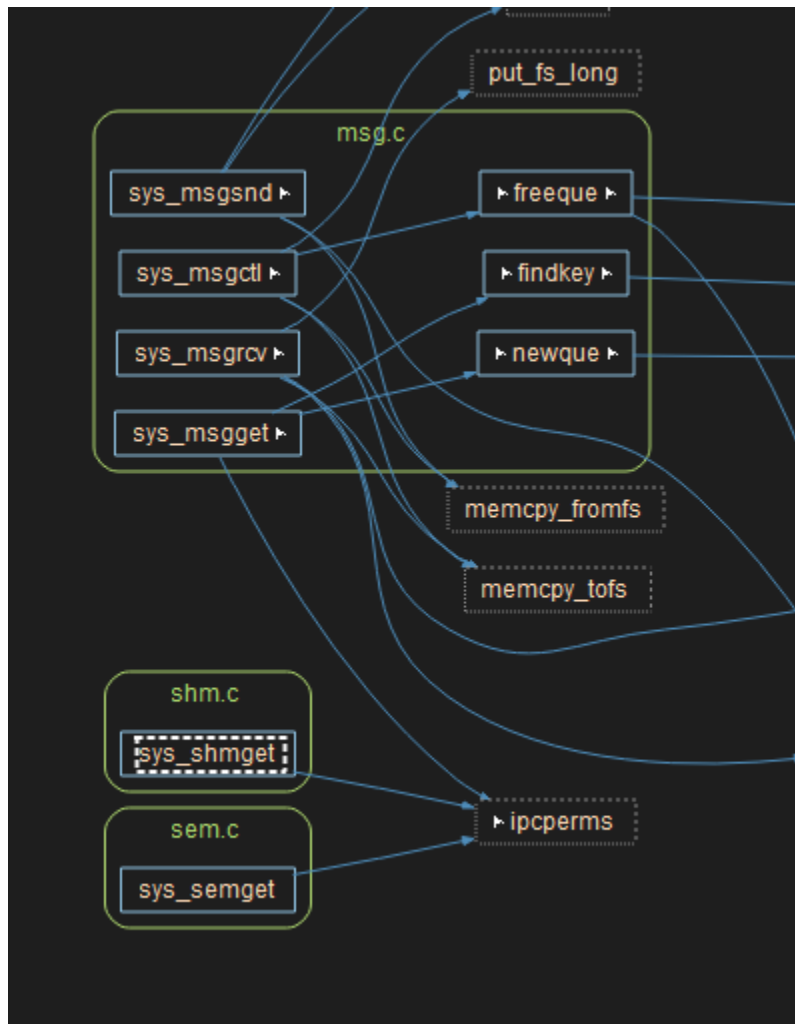
函数 `shm_swap_in` 功能：处理共享内存页的换入。从交换空间或文件读取页内容，更新页表项和内存统计。

函数 `shm_swap` 输入：`prio`：交换优先级

函数 `shm_swap` 输出：成功返回 1，失败返回 0

函数 `shm_swap` 功能：将共享内存页换出到交换空间。选择要换出的页，更新所有附加进程的页表，写入交换空间。

3. 程序流程图



Includes:

errno.h

sched.h

msg.h

stat.h

malloc.h

segment.h

File
msg.c

int findkey	struct msqid_ds *[128] msgque
int newque	int msgbytes
void freeque	int msghdrs
void msg_init	unsigned short msg_seq
int sys_msgsnd	int used_queues
int sys_msgrcv	int max_msqid
int sys_msgget	struct wait_queue * msg_lock
int sys_msgctl	

Includes:

errno.h

sched.h

mm.h

ipc.h

shm.h

stat.h

malloc.h

segment.h

pgtable.h

File
shm.c

int findkey

int newseg

void killseg

void insert_attach

void remove_attach

int shm_map

void shm_open

void shm_close

int shm_swap_in

void shm_init

int sys_shmget

int sys_shmctl

int sys_shmat

int sys_shmdt

int shm_swap

THREE_LEVEL =

int shm_tot

int shm_rss

int shm_swp

int max_shmid

struct wait_queue * shm_lock

struct shmseg_ds *[128] shm_segs

unsigned short shm_seq

ulong swap_attempts

ulong swap_successes

ulong used_segs

struct vm_operations_struct
shm_vm_ops

unsigned long swap_id

unsigned long swap_idx

Includes:

errno.h
segment.h
string.h
sched.h
sem.h
ipc.h
stat.h
malloc.h

File

sem.c

int findkey

int newary

void insert_into_queue

void remove_from_queue

int try_semop

int do_semop

void update_queue

int count_semncnt

int count_semzcnt

void freeary

void sem_init

int sys_semget

int sys_semctl

int sys_semop

void sem_exit

struct semid_ds *[128] semary

int used_sems

int used_semids

struct wait_queue * sem_lock

int max_semid

unsigned short sem_seq